

Leveraging LDP for High-Trust OpenSearch UBI

Keeping relevance work alive when you cannot store the queries

Jeff Zemerick

OpenSearchCon, San Jose
September 24, 2026

About me

- Independent consultant, Mountain Fog
- OpenSearch UBI and opensearch-migrations maintainer
- Apache Software Foundation member, OpenNLP PMC Chair
- PII redaction tooling at Philterd
- Works in search, NLP, and privacy

ACT 1

The blocker

To tune search, you have to see what users do

- Relevance work is empirical, not theoretical
- Which queries fire, which results get clicked, at which position
- Learning-to-rank trains on exactly this signal
- Without it you are guessing, and shipping guesses to production

OpenSearch UBI exists for this

- ubi_queries: user_query, query_id, client_id, timestamp
- ubi_events: action_name, object_id, position, session_id, page_id
- ubi.js collects interactions in the browser and ships them to the cluster
- This is the raw material relevance tuning runs on

Then legal reads the schema

- user_query is free text a person typed. It can contain anything
- client_id and session_id make it linkable across time
- In healthcare and finance, that combination is a hard stop
- Zero-trust stopped being a buzzword and became a blocker to relevance tuning

Redaction is reactive by construction

- You collect the PII first, then try to remove it
- You discover the failure after the fact, if at all
- A free-text query has no schema to redact against
- The fix has to be that the sensitive value never leaves the device readable

ACT 2

Embeddings are not anonymization

The obvious first idea

- "We will not store the text. We will store the embedding."
- It feels private. It is a vector of floats, not words
- It is not private. It is a lossy encoding
- Vector inversion recovers the intent

LIVE NOTEBOOK

Demo: invert a raw query vector

Notebook sections 1 to 6, then section 9

The raw vector gives up the intent

- Query: "laptop computer"
- Attacker inverts the raw vector, top hit: Laptop
- No text was stored, and the intent leaked anyway
- Storing embeddings is not a privacy control

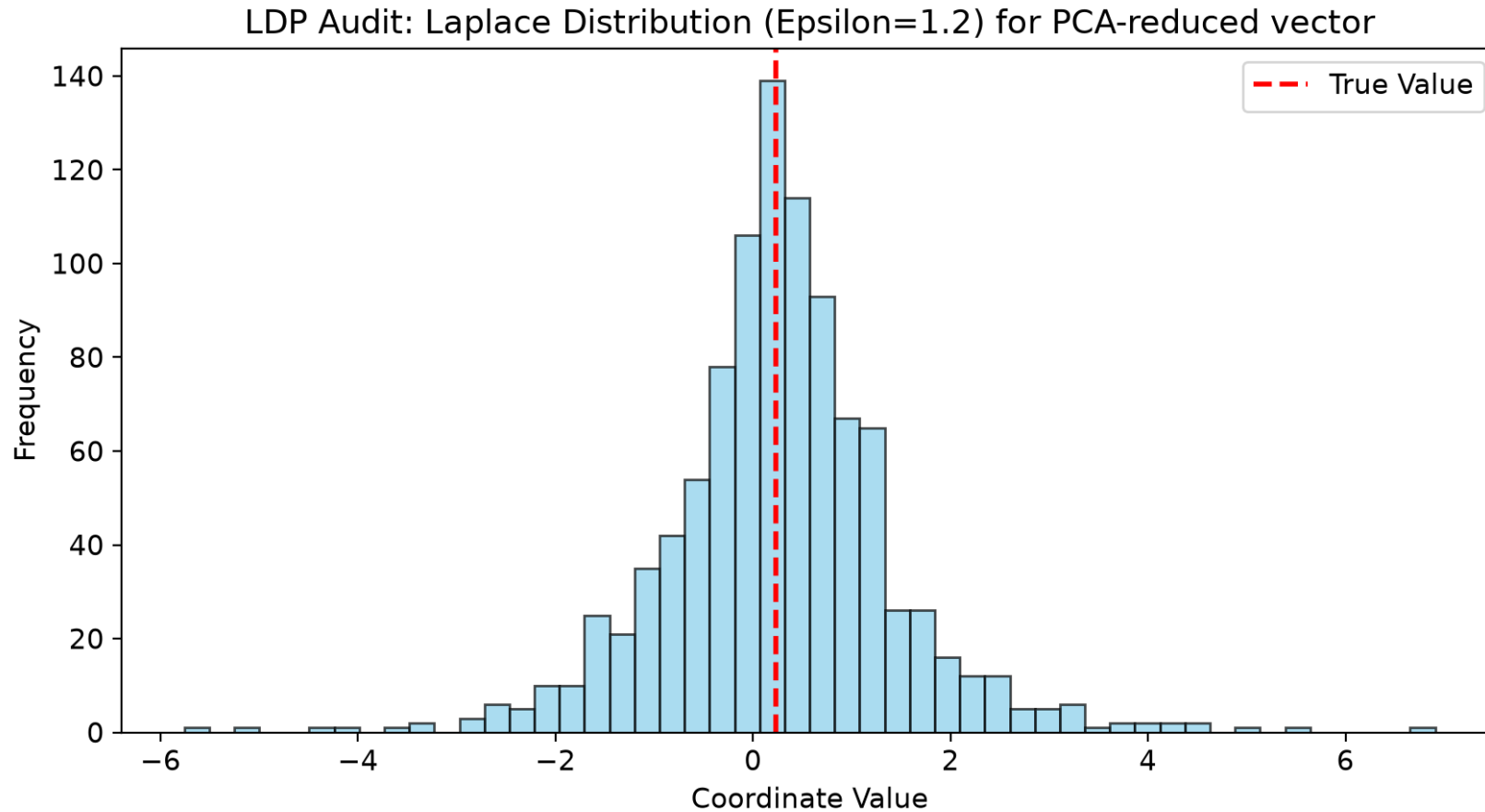
ACT 3

The mechanism, and how to verify it

Local Differential Privacy, in one slide

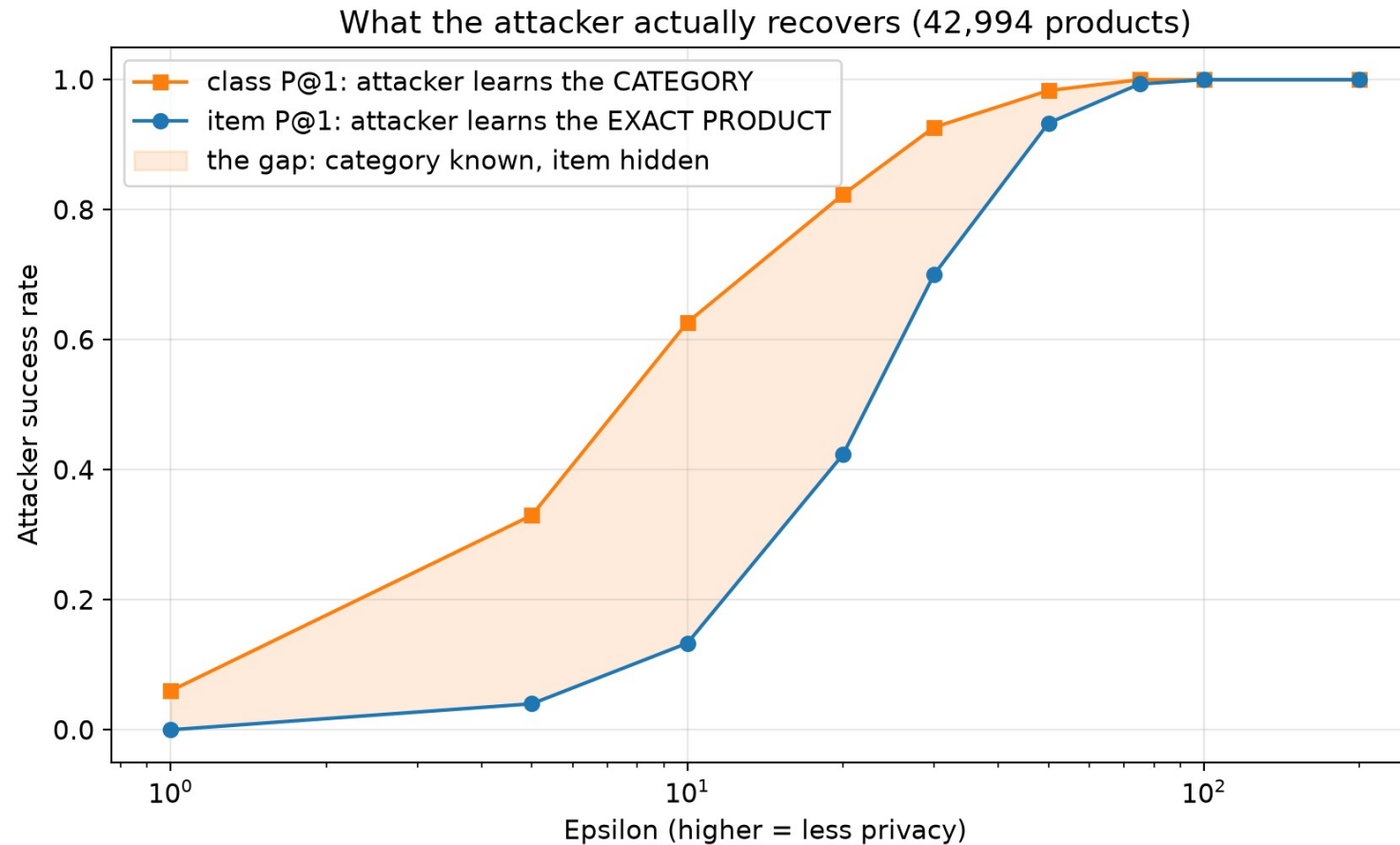
- Add calibrated Laplace noise to the query vector on the device
- Noise scale = sensitivity / epsilon
- Epsilon is the dial. Lower epsilon means more noise and more privacy
- The cluster never receives a readable query. There is nothing to scrub

Verification 1: the noise is what we claim



1,000 draws of the same query. The Laplace tent, centred on the true value.

Verification 2: measure an actual attacker



42,994 real Wayfair products. Attacker success against epsilon.

ACT 4

What it costs

That first demo was rigged

- It ran at epsilon 1.2 on a 20 document index
- Privacy was excellent. Utility was destroyed
- P@1 of 0.07 against a random baseline of 0.05
- The attacker learned nothing, and neither did the user

The tradeoff on the toy index

R@5 holds up long after P@1 collapses. The usable window is epsilon 3 to 5.

epsilon	P@1	R@5
1.2	0.07	0.45
3	0.38	0.75
5	0.72	0.95
10	0.98	1.00

20 documents. Random chance is P@1 0.05 and R@5 0.25.

A real index: the curves separate

The category leaks long before the item. At epsilon 10, 0.63 against 0.13.

epsilon	item P@1	class P@1
5	0.04	0.33
10	0.13	0.63
20	0.42	0.82
50	0.93	0.98

Chance is item 0.00002 and class 0.026. WANDS, 42,994 products, 861 classes.

ACT 5

What you keep

The asymmetry is the whole point

- Laplace noise is zero-mean
- It cancels when you average over many independent users
- One person's query is unrecoverable
- The trend across ten thousand people is not

LIVE NOTEBOOK

Demo: cohorts of real users

Notebook section 11

480 real WANDS queries, grouped into 5 cohorts

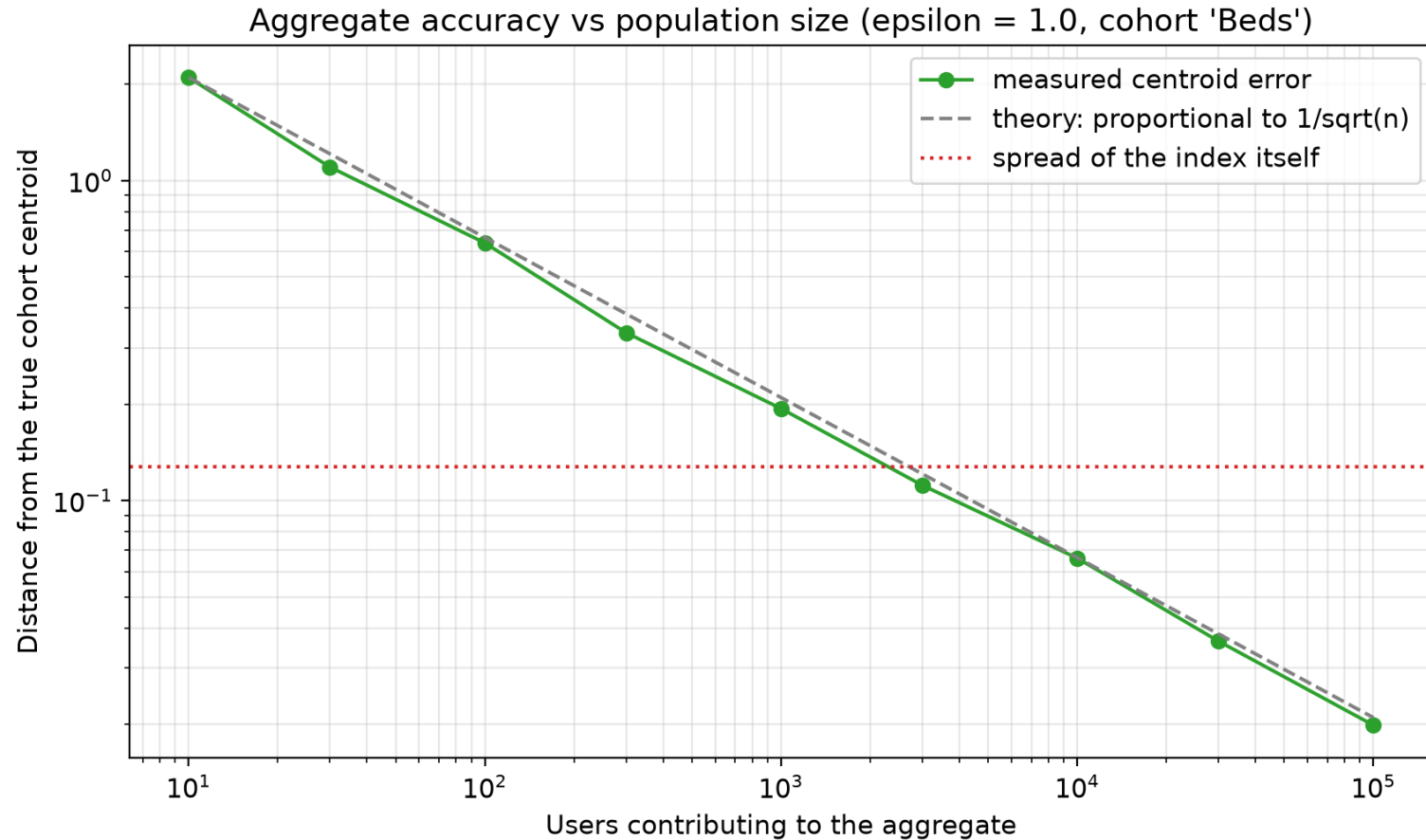
Every user privatizes independently, on their own device

Individuals hide. The crowd does not.

cohort	individual	chance	cohort ID
Wall Art	0.020	0.014	OK
Accent Chairs	0.020	0.027	OK
Beds	0.063	0.026	OK
Area Rugs	0.070	0.031	OK
Coffee & Cocktail Tables	0.030	0.025	OK

epsilon 1.0, heavier noise than anything in Act 4. 20,000 users per cohort. 5 of 5 recovered.

Error falls as $1 / \sqrt{n}$, exactly as theory says



Doubling the privacy budget is expensive. Four times the users is free.

The number to take home

- Error drops below the spread of the index at roughly 2,500 users
- Segments with thousands of users are measurable under LDP
- Segments with dozens are not
- Your head terms work. Your long tail does not

LDP does not cost you your analytics.

It costs you the ability to ask about one person.

ACT 6

Back to OpenSearch, and the limits

Where the noise gets injected

- In ubi.js, on the device, before the event is sent
- The cluster stores noised vectors and never sees the raw query
- Aggregation happens at query time over the noised data
- Nothing downstream needs to be trusted, because nothing downstream has the original

What your dashboards can still compute

- Aggregate query intent per segment
- Demand trends and shifts over time
- Cohort-level training signal for learning-to-rank
- What they cannot do is show you one user's session

The limits, stated plainly

- The category leaks. Harmless for furniture, maybe not for a medical corpus
- Epsilon does not transfer between indexes. It depends on coordinate spread
- Low-traffic segments stay unmeasurable
- Clicks are counts, not vectors, and want randomized response instead

Takeaways

- Storing embeddings instead of text is not privacy
- Noise at the source beats scrubbing after the fact
- Verify the mechanism by attacking it, not by trusting it
- You trade individual resolution for population accuracy, and relevance only needs the latter

Questions

github.com/jzonthemtn/ldp-for-search

Jeff Zemerick

OpenSearchCon, San Jose
September 24, 2026